

List of Experiments (Set 11 - Set 20)

DSA06 - Data Handling and Visualization

List of Experiments (Set 11 to Set 20)

Set 11: Product Category Analysis

Dataset: Product Categories

Category	Sales (in \$)
Electronics	50000
Clothing	35000
Appliances	40000

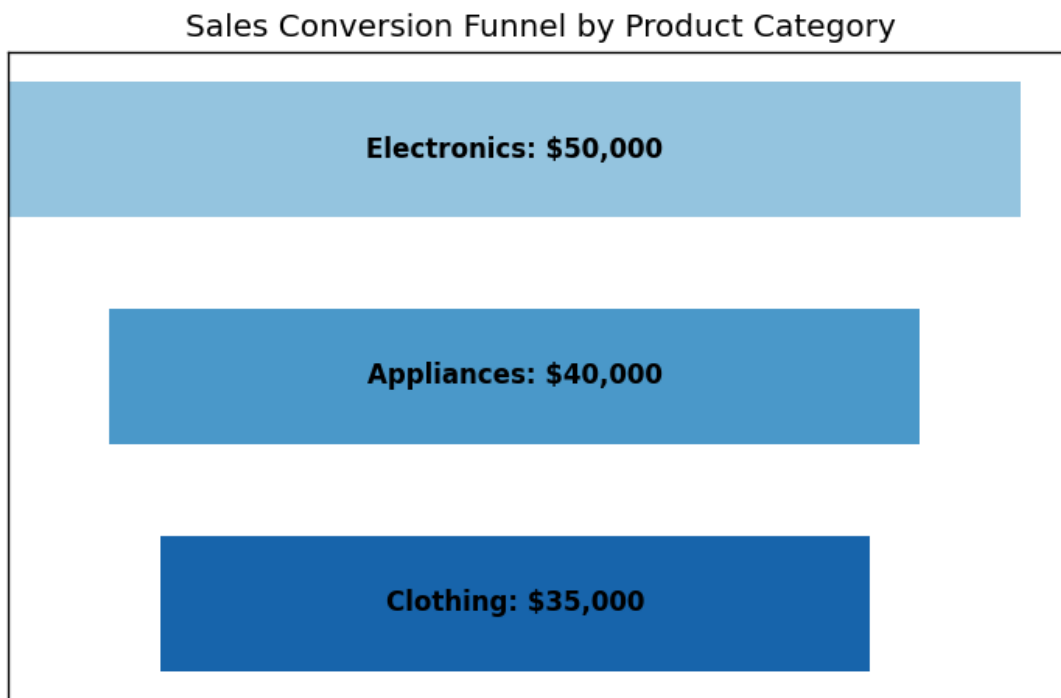
Q1. Funnel chart to analyze the sales conversion process

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Category": ["Electronics", "Appliances", "Clothing"],
    "Sales (in $)": [50000, 40000, 35000]
}).sort_values("Sales (in $)", ascending=False).reset_index(drop=True)

max_val = df["Sales (in $)"].max()
for i, row in df.iterrows():
    width = row["Sales (in $)"]
    left = (max_val - width) / 2
    plt.barh(i, width, left=left, height=0.6)
    plt.text(max_val/2, i, f'{row["Category"]}: ${row["Sales (in $)"]:,}',
             ha="center", va="center")
plt.yticks([]); plt.xticks([])
plt.title("Sales Conversion Funnel by Product Category")
plt.gca().invert_yaxis()
plt.show()
```

Output:



Set 11 Q1

Q2. Table to display sales data for each product category

```
print(df)
```

Output:

Product Category Sales Data

Category	Sales (in \$)
Electronics	50000
Appliances	40000
Clothing	35000

Set 11 Q2

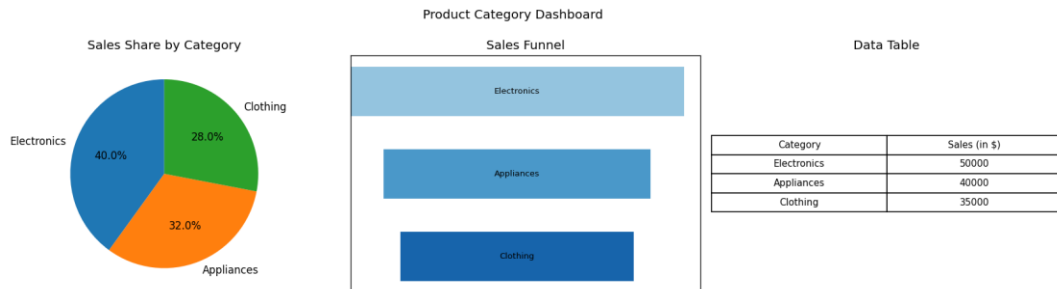
Q3. Dashboard combining pie chart, funnel chart and table

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

```
# axes[0]: pie chart, axes[1]: funnel chart, axes[2]: data table
```

```
plt.suptitle("Product Category Dashboard")
plt.show()
```

Output:



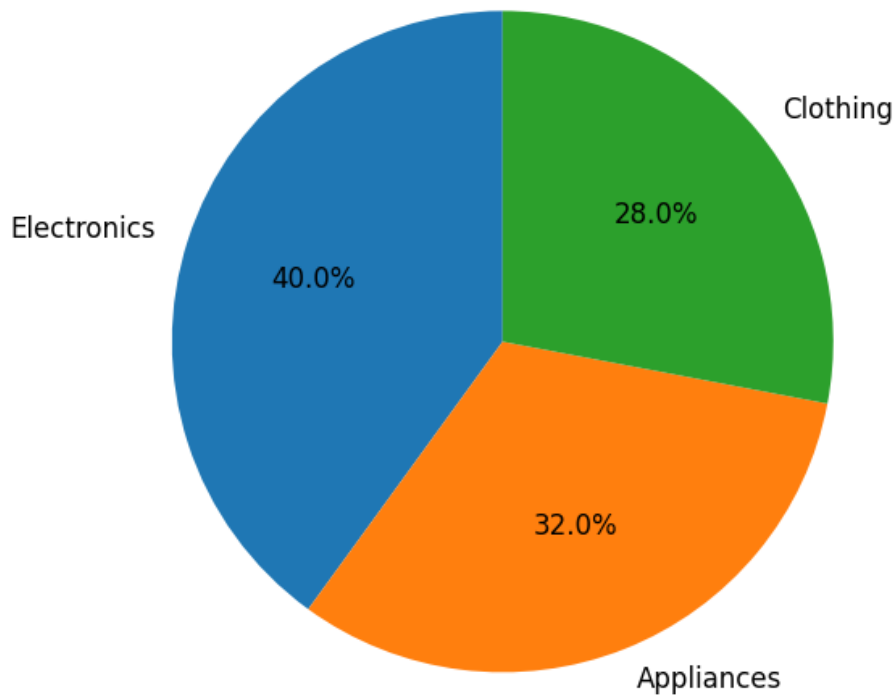
Set 11 Q3

Q4. Pie chart to represent distribution of sales across product categories

```
plt.pie(df["Sales (in $)"], labels=df["Category"], autopct="%1.1f%%",
startangle=90)
plt.title("Distribution of Sales Across Product Categories")
plt.show()
```

Output:

Distribution of Sales Across Product Categories



Set 11 Q4

Set 12: Website Analytics

Dataset: Website Traffic

Date	Page Views	Click-through Rate
2023-01-01	1500	2.3%
2023-01-02	1600	2.7%
2023-01-03	1400	2.0%

Q1. Line chart to visualize trend in daily page views

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.DataFrame({
    "Date": ["2023-01-01", "2023-01-02", "2023-01-03"],
    "Page Views": [1500, 1600, 1400],
```

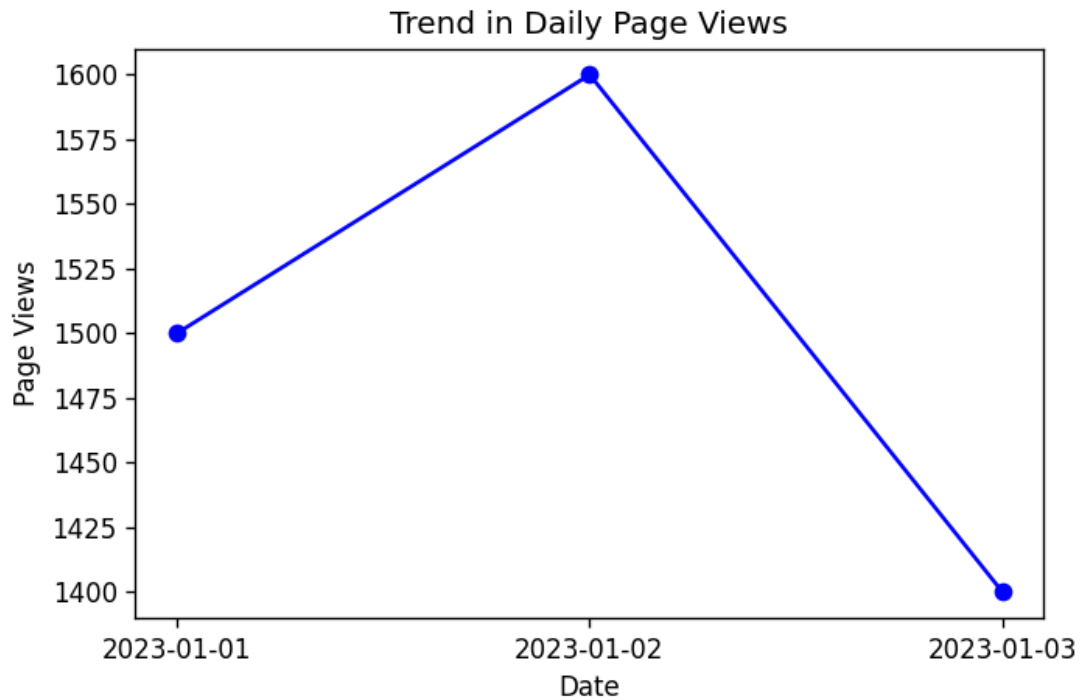
```

    "Click-through Rate": [2.3, 2.7, 2.0]
})

plt.plot(df["Date"], df["Page Views"], marker="o", color="blue")
plt.xlabel("Date"); plt.ylabel("Page Views")
plt.title("Trend in Daily Page Views")
plt.show()

```

Output:



Set 12 Q1

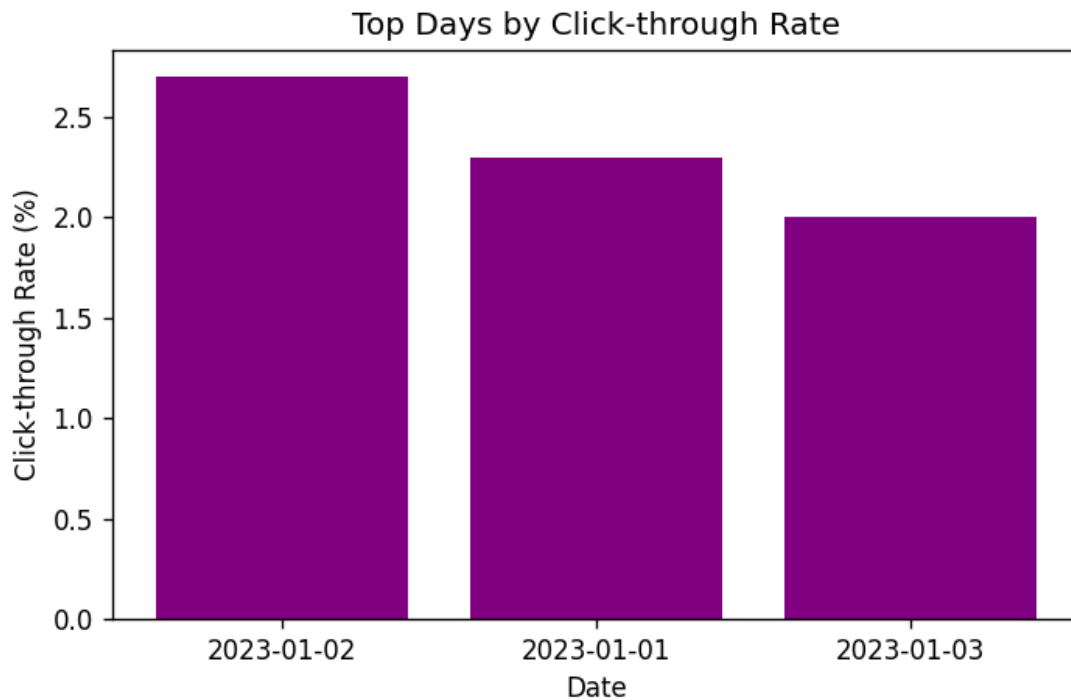
Q2. Bar chart showing top N days with highest CTR

```

top_n = df.sort_values("Click-through Rate", ascending=False)
plt.bar(top_n["Date"], top_n["Click-through Rate"], color="purple")
plt.xlabel("Date"); plt.ylabel("Click-through Rate (%)")
plt.title("Top Days by Click-through Rate")
plt.show()

```

Output:



Set 12 Q2

Q3. Table to show website traffic data

```
print(df)
```

Output:

Website Traffic Data

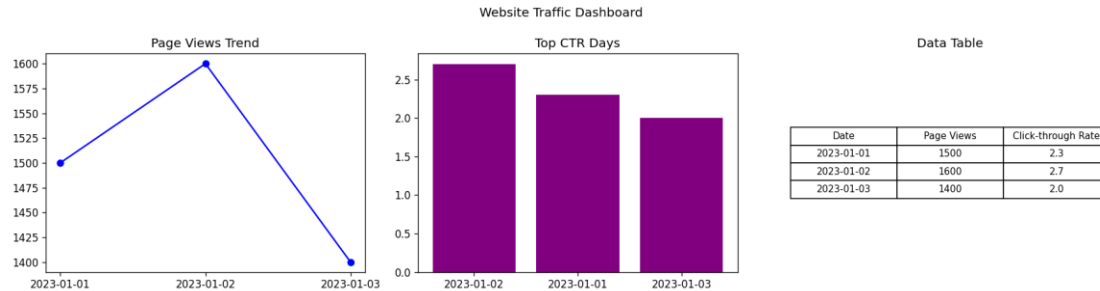
Date	Page Views	Click-through Rate
2023-01-01	1500	2.3
2023-01-02	1600	2.7
2023-01-03	1400	2.0

Set 12 Q3

Q4. Dashboard combining line chart, bar chart and table

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
# axes[0]: line chart, axes[1]: bar chart, axes[2]: data table
plt.suptitle("Website Traffic Dashboard")
plt.show()
```

Output:



Set 12 Q4

Set 13: Geographic Data

Dataset: Geographic Data

City	Population	Avg. Temperature	Elevation
City A	500000	75	1000
City B	700000	68	800
City C	600000	80	1200

Q1. Map chart to visualize distribution of cities

(No latitude/longitude was given in the dataset, so the cities are placed on a simple illustrative layout, bubble size = population.)

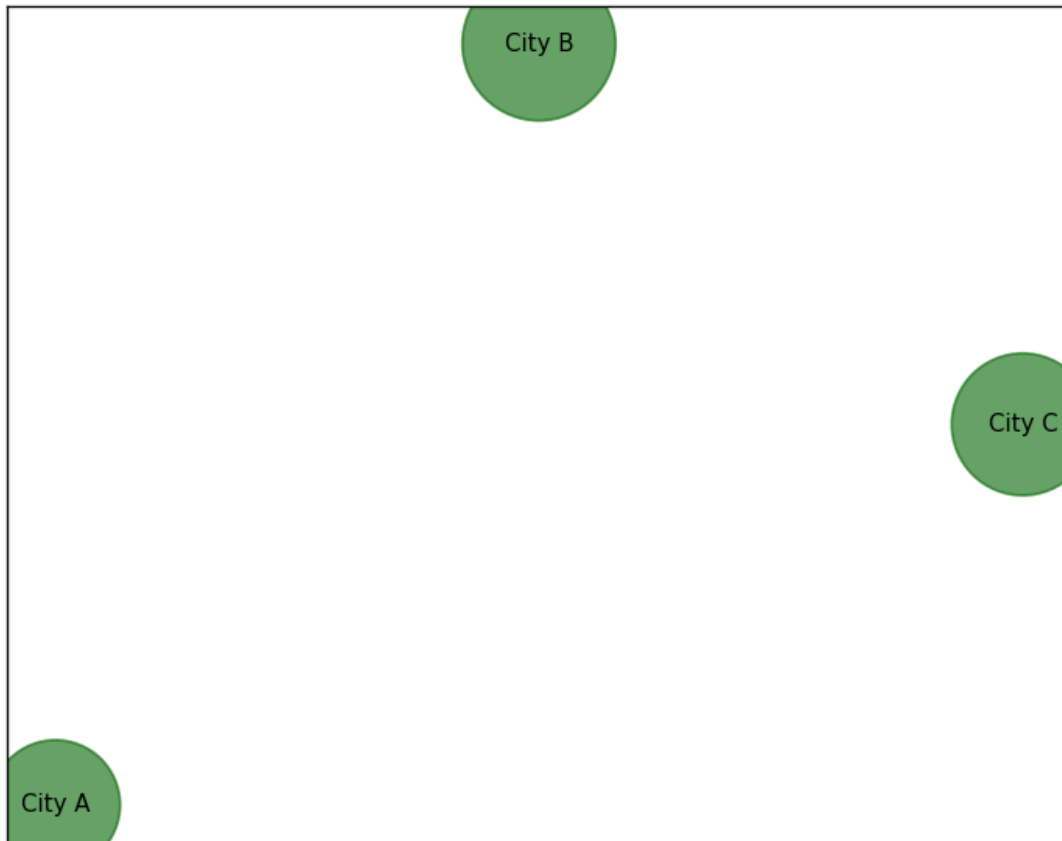
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "City": ["City A", "City B", "City C"],
    "Population": [500000, 700000, 600000],
    "Avg. Temperature": [75, 68, 80],
    "Elevation": [1000, 800, 1200]
})

x_pos, y_pos = [1, 2, 3], [1, 2, 1.5]
plt.scatter(x_pos, y_pos, s=df["Population"]/200, alpha=0.6,
color="darkgreen")
for i, city in enumerate(df["City"]):
    plt.text(x_pos[i], y_pos[i], city, ha="center", va="center")
plt.title("City Map (bubble size = Population)")
plt.xticks([]); plt.yticks([])
plt.show()
```

Output:

City Map (bubble size = Population)

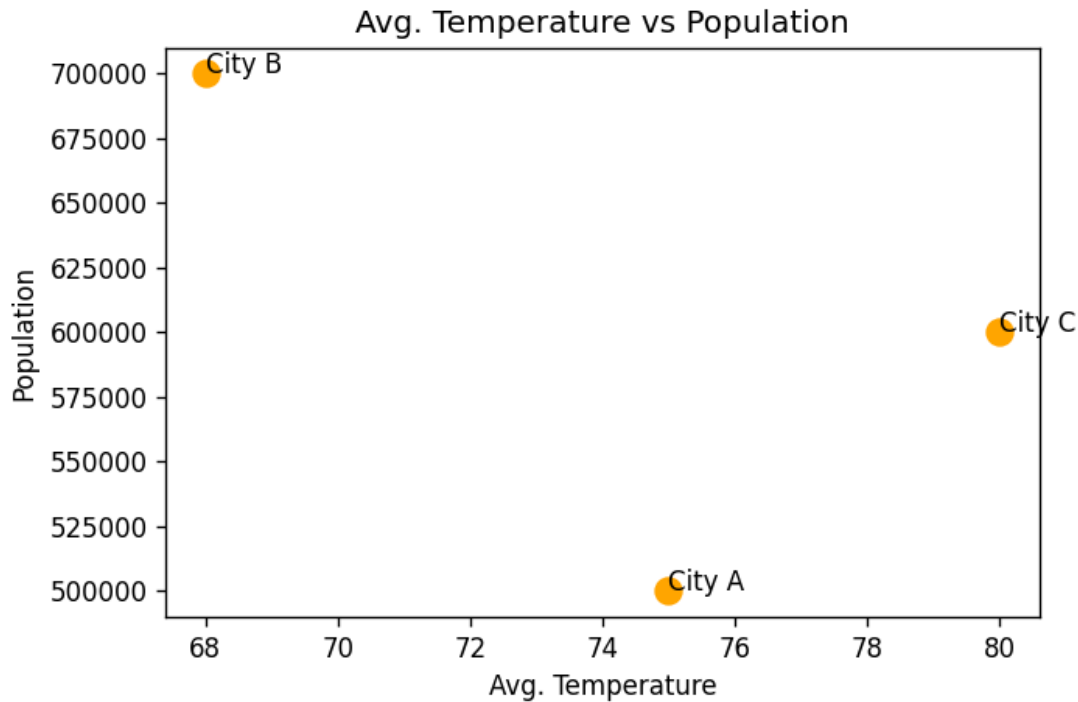


Set 13 Q1

Q2. Scatter plot: Avg. Temperature vs Population

```
plt.scatter(df["Avg. Temperature"], df["Population"], color="orange", s=100)
for i, city in enumerate(df["City"]):
    plt.annotate(city, (df["Avg. Temperature"][i], df["Population"][i]))
plt.xlabel("Avg. Temperature"); plt.ylabel("Population")
plt.title("Avg. Temperature vs Population")
plt.show()
```

Output:



Set 13 Q2

Insight: City C has the highest average temperature and a mid-range population, while City B has the largest population but the lowest average temperature — no strong direct correlation is seen between temperature and population size.

Q3. Table to display geographic data

```
print(df)
```

Output:

Geographic Data

City	Population	Avg. Temperature	Elevation
City A	500000	75	1000
City B	700000	68	800
City C	600000	80	1200

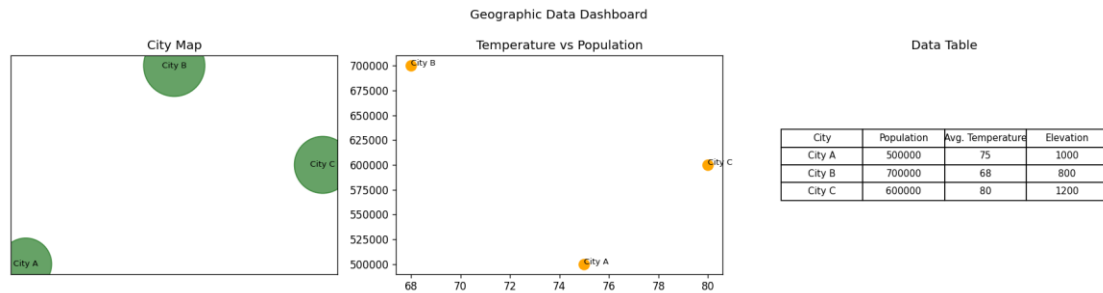
Set 13 Q3

Q4. Dashboard combining map chart, scatter plot and table

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))  
# axes[0]: map/bubble chart, axes[1]: scatter plot, axes[2]: data table
```

```
plt.suptitle("Geographic Data Dashboard")
plt.show()
```

Output:



Set 13 Q4

Set 14: Survey Results

Dataset: Survey Results

Respondent	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

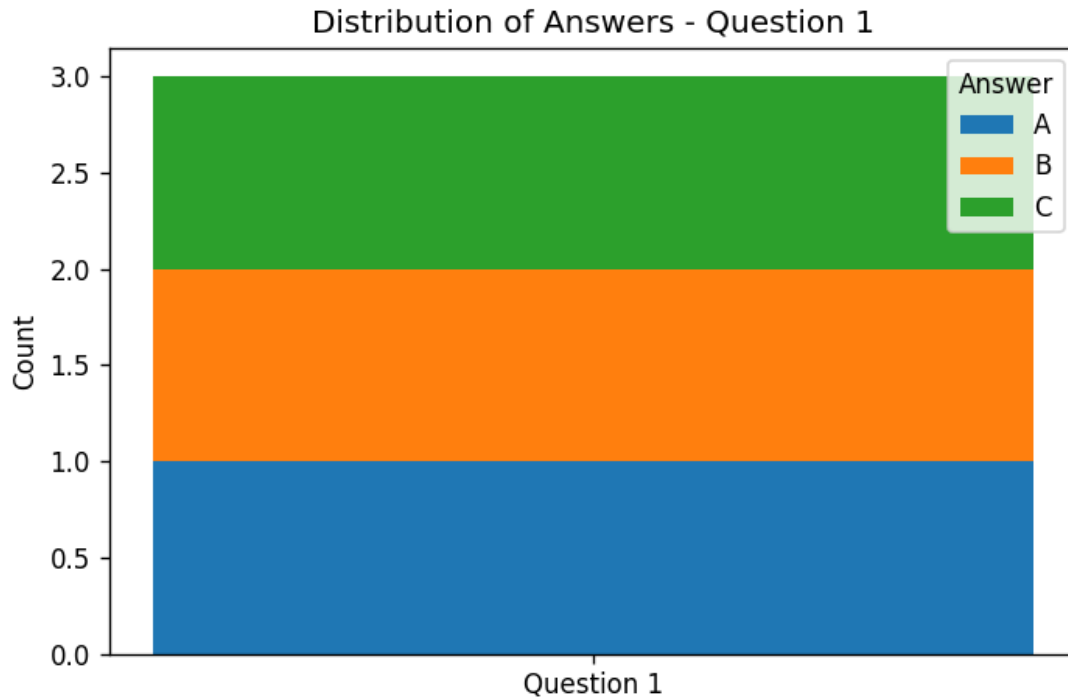
Q1. Stacked bar chart for distribution of answers for Question 1

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Respondent": [1, 2, 3],
    "Question 1": ["A", "B", "C"],
    "Question 2": ["B", "A", "A"],
    "Question 3": ["C", "D", "B"]
})

q1_counts = df["Question 1"].value_counts().sort_index()
bottom = 0
for ans, cnt in q1_counts.items():
    plt.bar("Question 1", cnt, bottom=bottom, label=ans)
    bottom += cnt
plt.ylabel("Count")
plt.title("Distribution of Answers - Question 1")
plt.legend(title="Answer")
plt.show()
```

Output:



Set 14 Q1

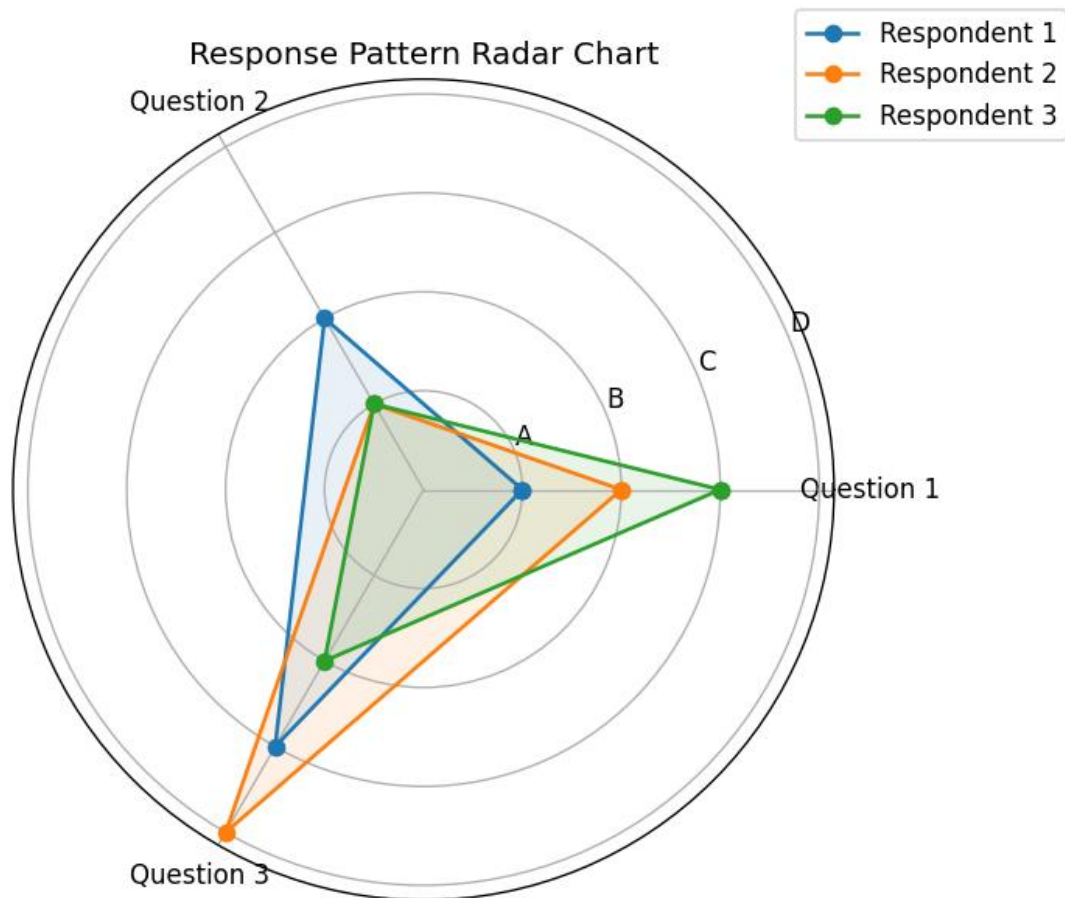
Q2. Radar chart for overall pattern of responses

```
import numpy as np

answer_map = {"A": 1, "B": 2, "C": 3, "D": 4}
questions = ["Question 1", "Question 2", "Question 3"]
angles = np.linspace(0, 2*np.pi, len(questions), endpoint=False).tolist()
angles += angles[:1]

ax = plt.subplot(111, polar=True)
for i, row in df.iterrows():
    values = [answer_map[row[q]] for q in questions]
    values += values[:1]
    ax.plot(angles, values, marker="o", label=f'Respondent {row["Respondent"]}')
    ax.fill(angles, values, alpha=0.1)
ax.set_xticks(angles[:-1]); ax.set_xticklabels(questions)
plt.title("Response Pattern Radar Chart")
plt.legend()
plt.show()
```

Output:



Set 14 Q2

Q3. Table to show survey response data

```
print(df)
```

Output:

Survey Response Data

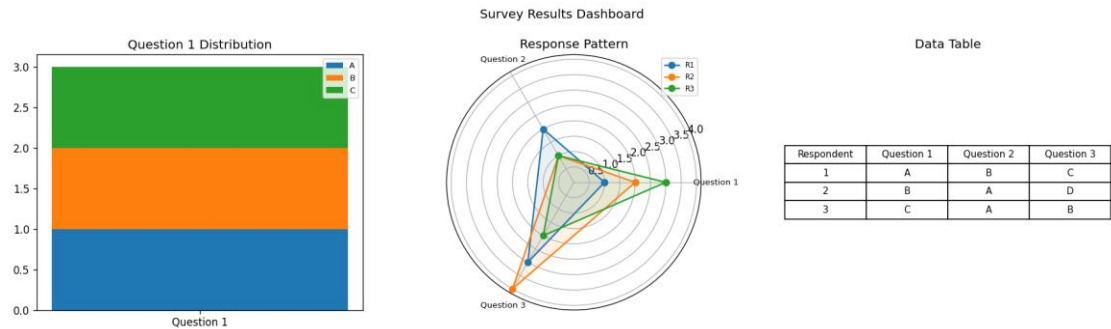
Respondent	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

Set 14 Q3

Q4. Dashboard combining stacked bar chart, radar chart and table

```
fig = plt.figure(figsize=(15, 4.5))
# subplot 1: stacked bar chart, subplot 2: radar chart, subplot 3: data table
plt.suptitle("Survey Results Dashboard")
plt.show()
```

Output:

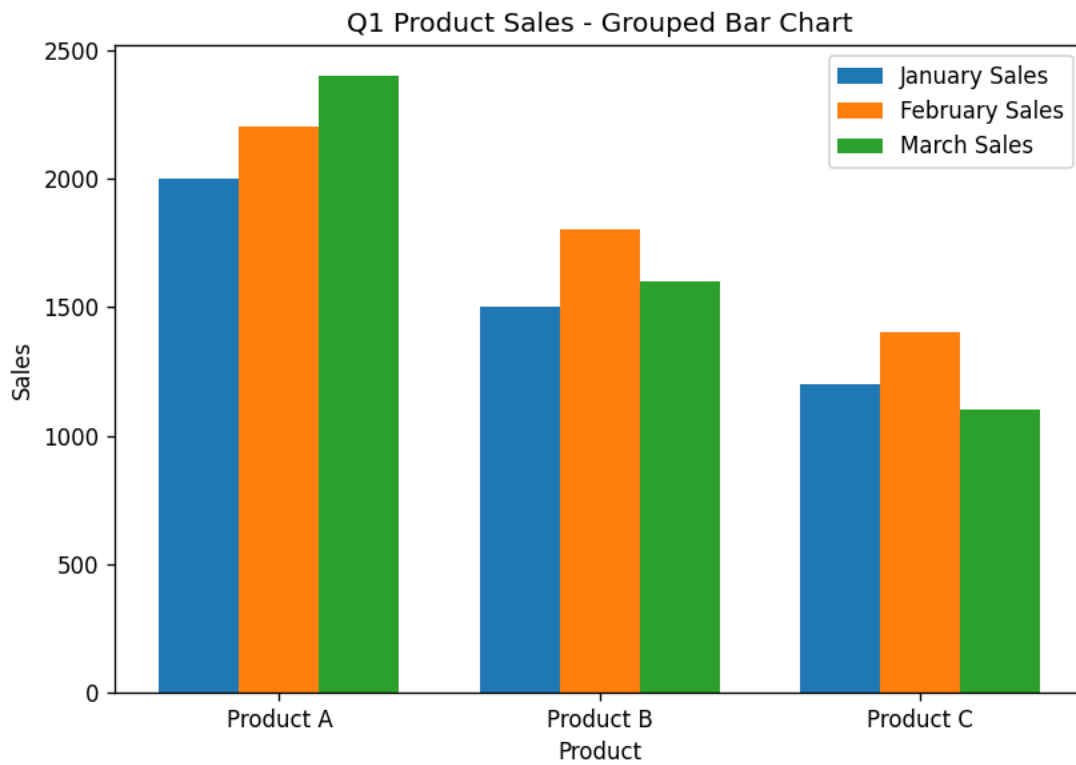


Set 14 Q4

Set 15: Product Sales Analysis

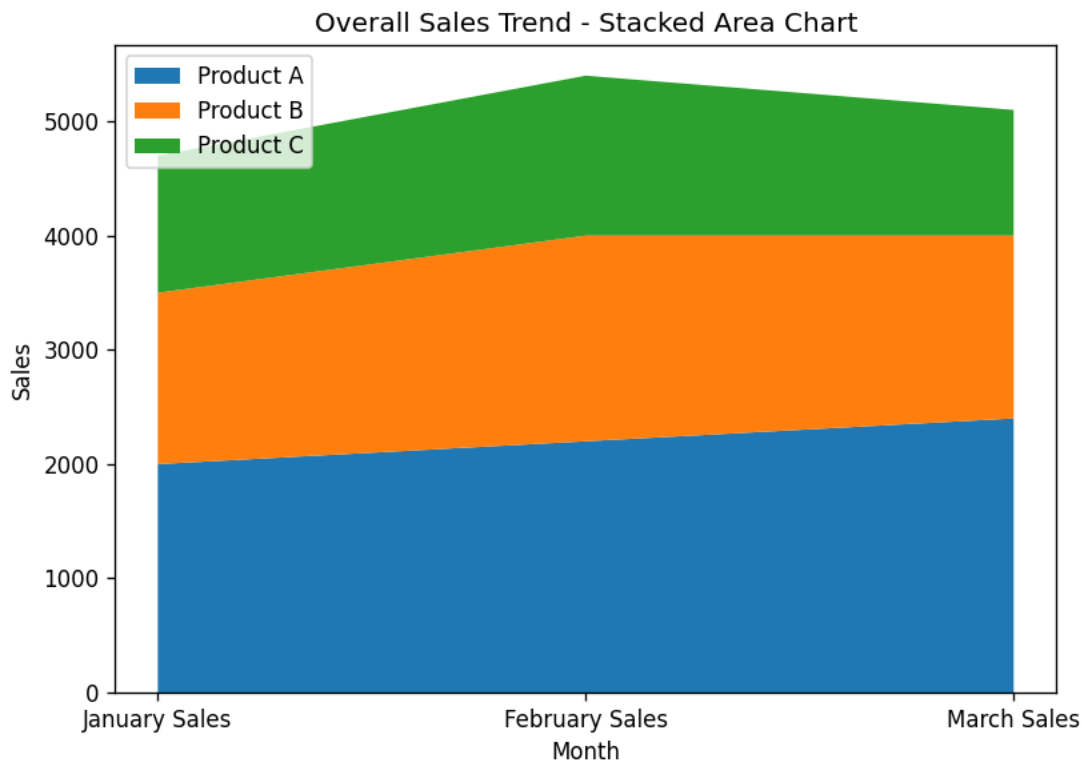
(Same dataset and tasks as Set 6.)

Q1. Grouped bar chart for Q1 product sales



Set 15 Q1

Q2. Stacked area chart for overall sales trend



Set 15 Q2

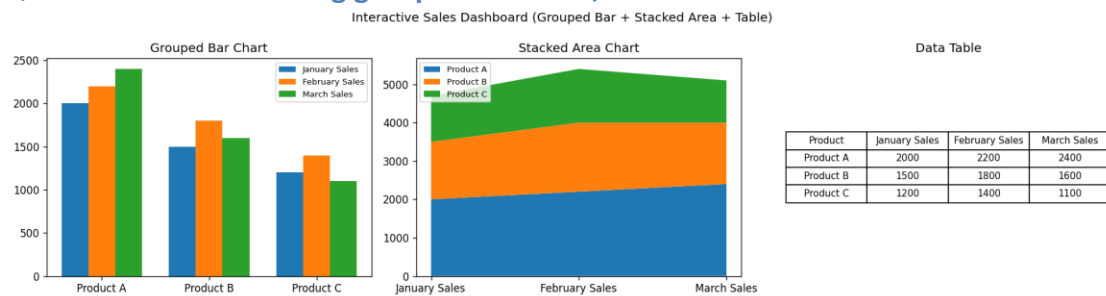
Q3. Table showing monthly sales data

Monthly Sales Data Table

Product	January Sales	February Sales	March Sales
Product A	2000	2200	2400
Product B	1500	1800	1600
Product C	1200	1400	1100

Set 15 Q3

Q4. Dashboard combining grouped bar chart, stacked area chart and table



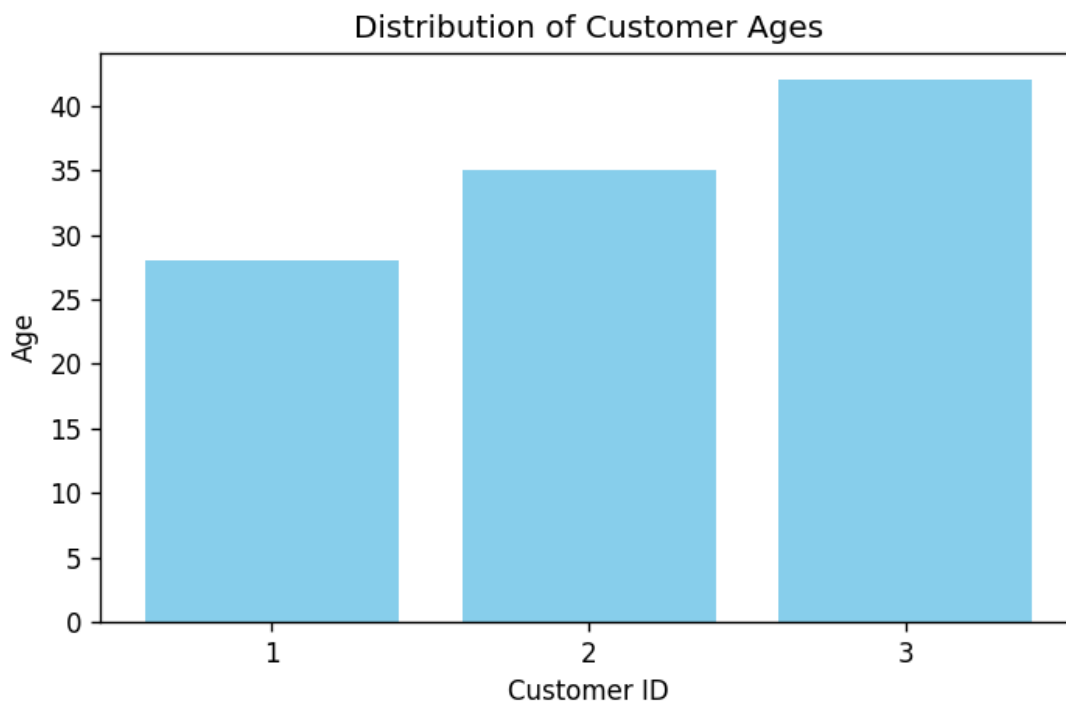
Set 15 Q4

(Code is identical to Set 6 — see Set 6 above.)

Set 16: Customer Demographics Analysis

(Same dataset and tasks as Set 7.)

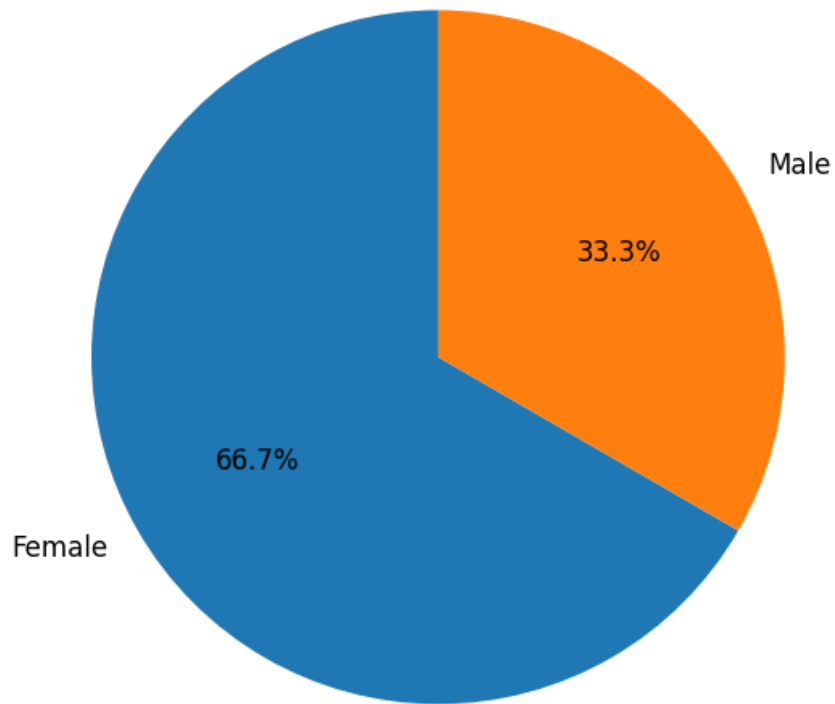
Q1. Bar chart for distribution of customer ages



Set 16 Q1

Q2. Pie chart for distribution of customers by gender

Customer Gender Distribution



Set 16 Q2

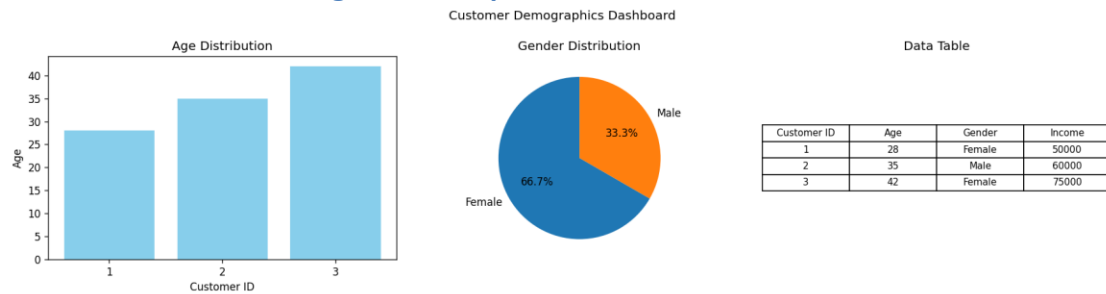
Q3. Table showing demographic information

Customer Demographic Data

Customer ID	Age	Gender	Income
1	28	Female	50000
2	35	Male	60000
3	42	Female	75000

Set 16 Q3

Q4. Dashboard combining bar chart, pie chart and table



Set 16 Q4

(Code is identical to Set 7 — see Set 7 above.)

Set 17: Employee Performance Analysis

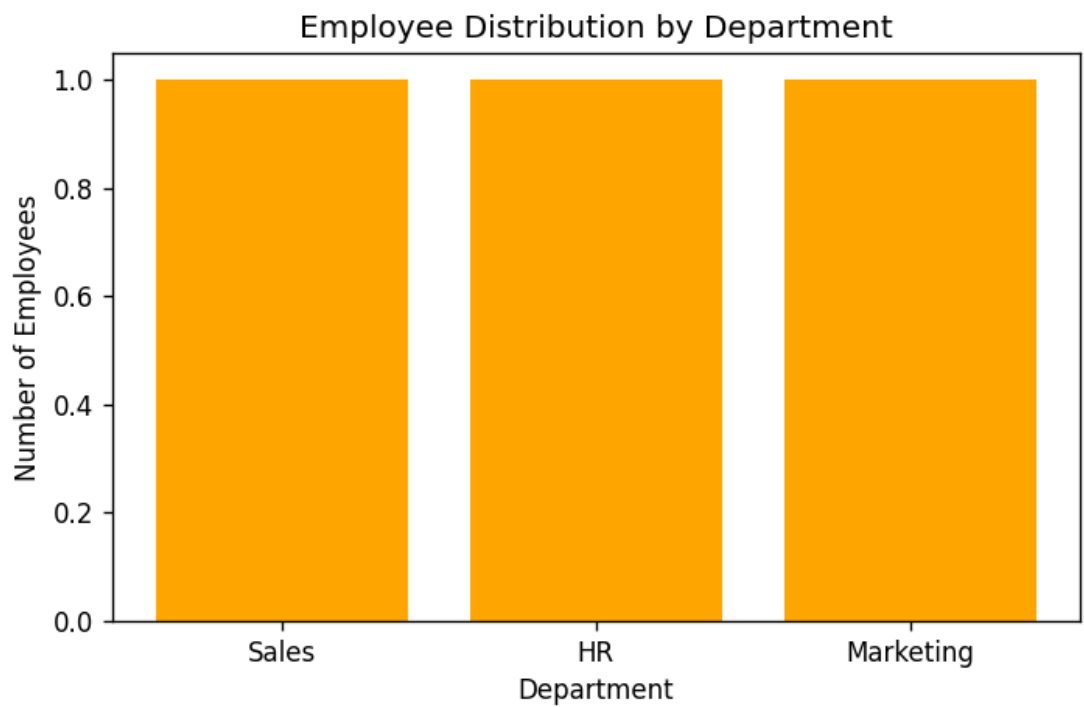
(Same dataset and tasks as Set 8.)

Q1. Line chart for performance trend of employees



Set 17 Q1

Q2. Bar chart for distribution of employees across departments



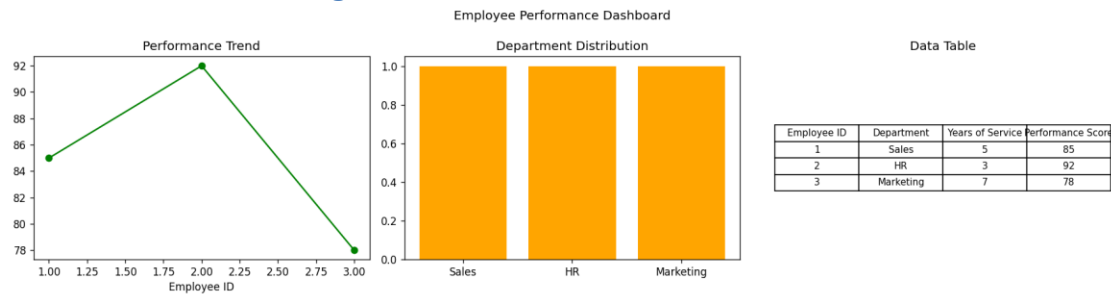
Set 17 Q2

Q3. Table displaying performance data

Employee Performance Data			
Employee ID	Department	Years of Service	Performance Score
1	Sales	5	85
2	HR	3	92
3	Marketing	7	78

Set 17 Q3

Q4. Dashboard combining line chart, bar chart and table



Set 17 Q4

(Code is identical to Set 8 — see Set 8 above.)

Set 18: Product Inventory Management

Dataset: Product Inventory

Product ID	Product Name	Quantity Available	Price (in \$)
1	Product A	250	20
2	Product B	175	15
3	Product C	300	18

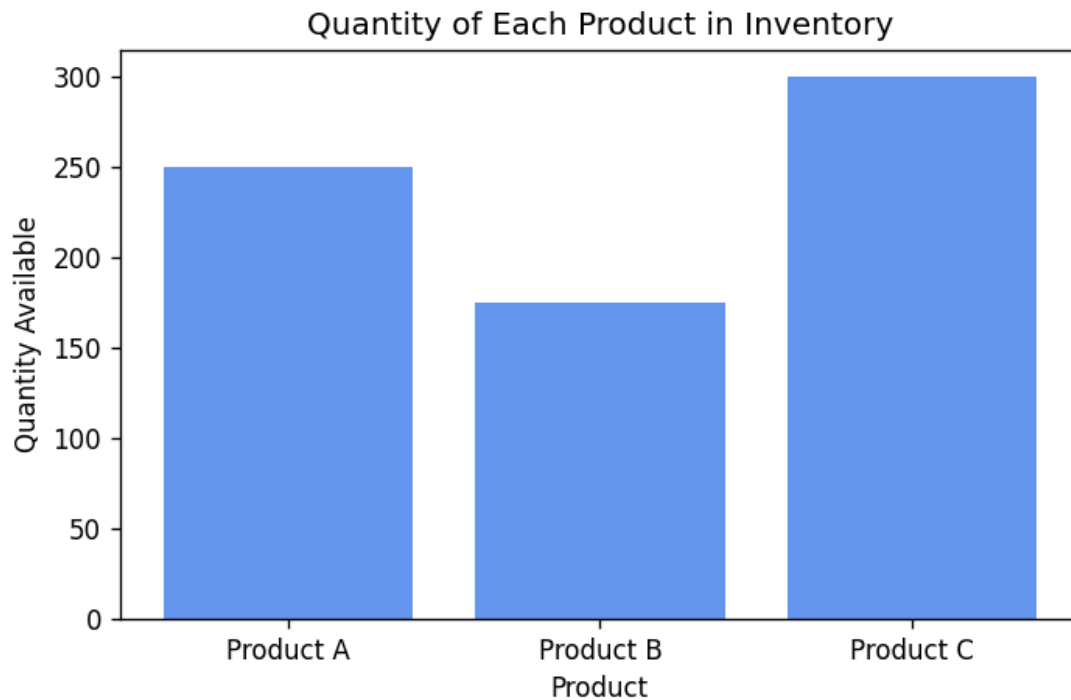
Q1. Bar chart to visualize quantity of each product

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Product ID": [1, 2, 3],
    "Product Name": ["Product A", "Product B", "Product C"],
    "Quantity Available": [250, 175, 300],
    "Price": [20, 15, 18]
})

plt.bar(df["Product Name"], df["Quantity Available"], color="cornflowerblue")
plt.xlabel("Product"); plt.ylabel("Quantity Available")
plt.title("Quantity of Each Product in Inventory")
plt.show()
```

Output:

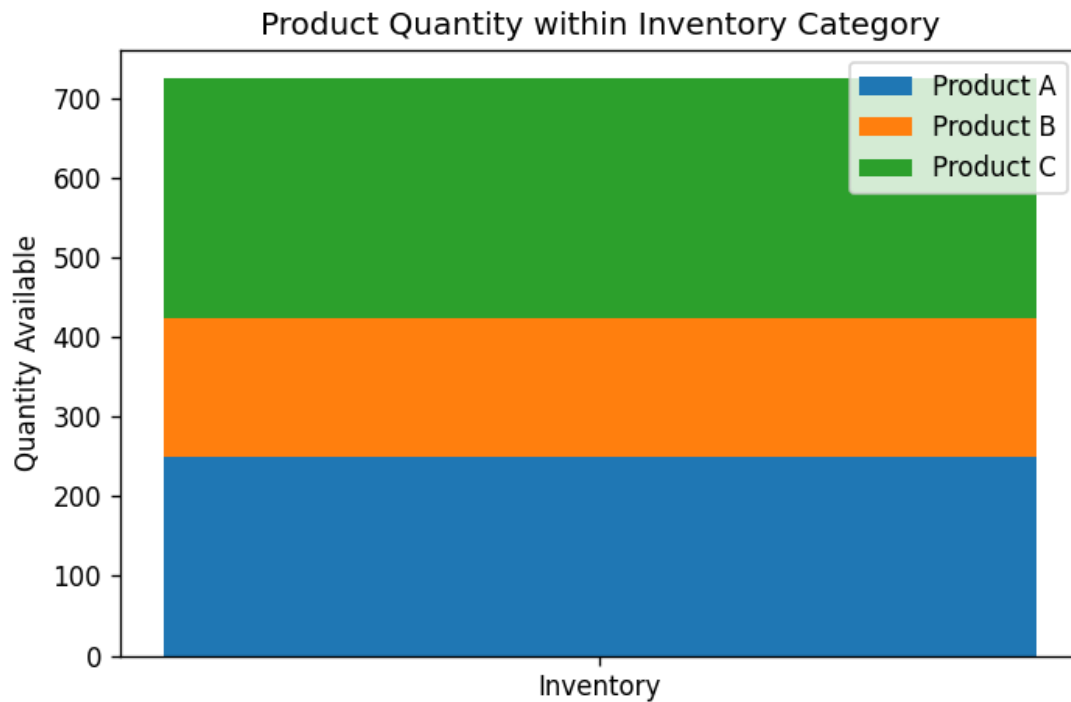


Set 18 Q1

Q2. Stacked bar chart to show quantity within product categories

```
bottom = 0
for i, row in df.iterrows():
    plt.bar("Inventory", row["Quantity Available"], bottom=bottom,
    label=row["Product Name"])
    bottom += row["Quantity Available"]
plt.ylabel("Quantity Available")
plt.title("Product Quantity within Inventory Category")
plt.legend()
plt.show()
```

Output:



Set 18 Q2

Q3. Table to show inventory data

```
print(df)
```

Output:

Product Inventory Data

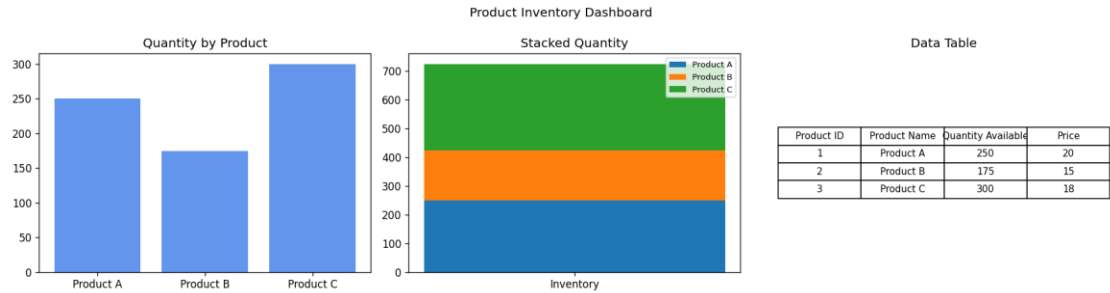
Product ID	Product Name	Quantity Available	Price
1	Product A	250	20
2	Product B	175	15
3	Product C	300	18

Set 18 Q3

Q4. Dashboard combining bar chart, stacked bar chart and table

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
# axes[0]: bar chart, axes[1]: stacked bar chart, axes[2]: data table
plt.suptitle("Product Inventory Dashboard")
plt.show()
```

Output:

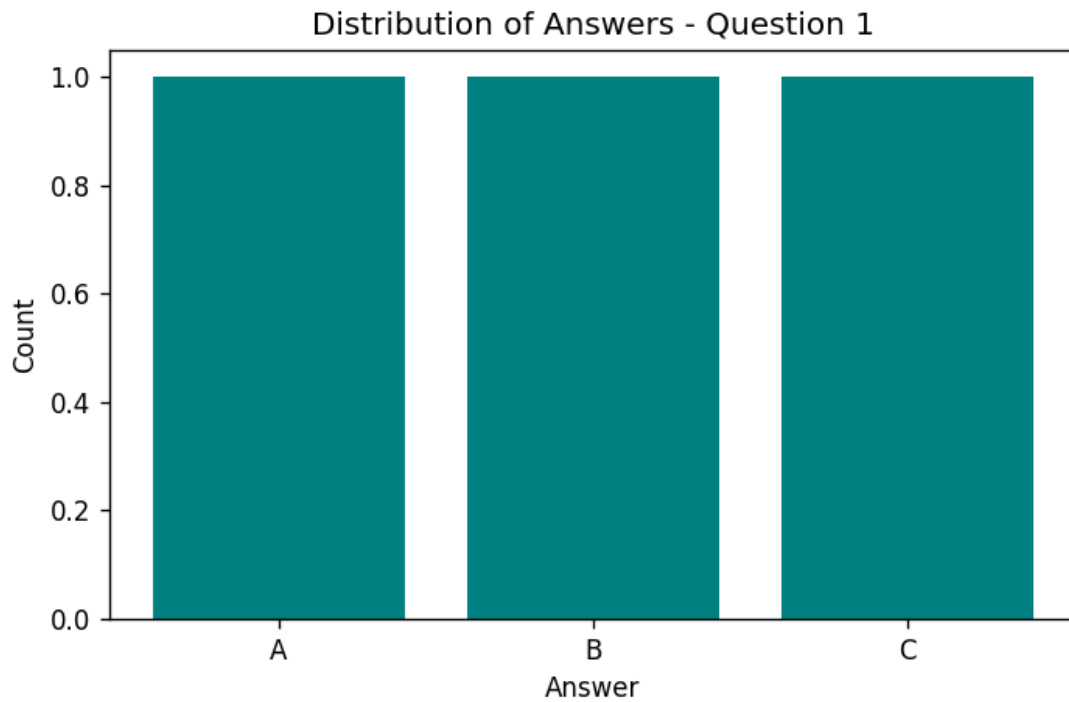


Set 18 Q4

Set 19: Survey Responses Analysis

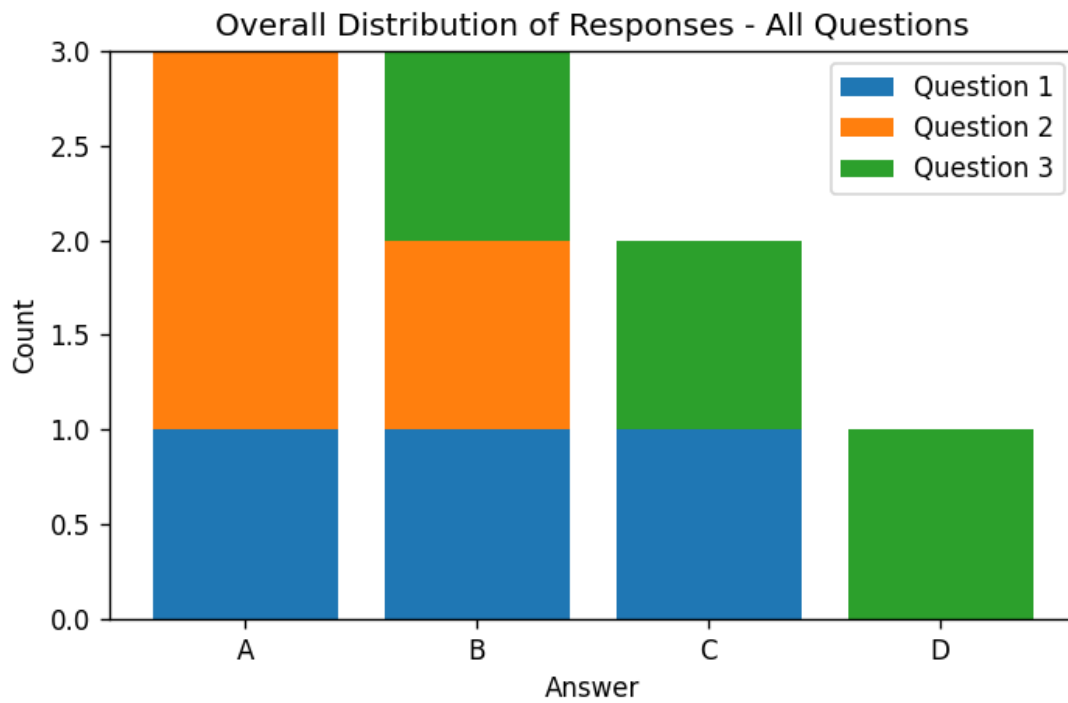
(Same dataset and tasks as Set 10.)

Q1. Grouped bar chart for distribution of answers for Question 1



Set 19 Q1

Q2. Stacked bar chart for overall distribution of responses



Set 19 Q2

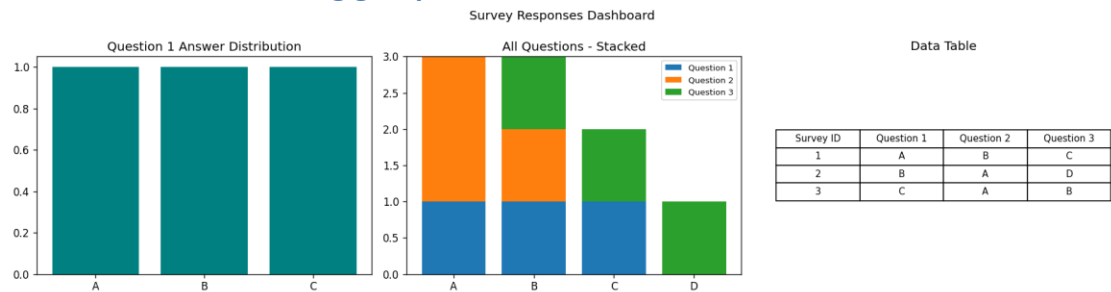
Q3. Table showing survey response data

Survey Response Data

Survey ID	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

Set 19 Q3

Q4. Dashboard combining grouped bar, stacked bar and table



Set 19 Q4

(Code is identical to Set 10 — see Set 10 above.)

Set 20: Stock Analysis

Dataset: Stock Prices

Date	Stock A	Stock B	Stock C
2023-01-01	100	150	120
2023-01-02	105	152	118
2023-01-03	110	148	122

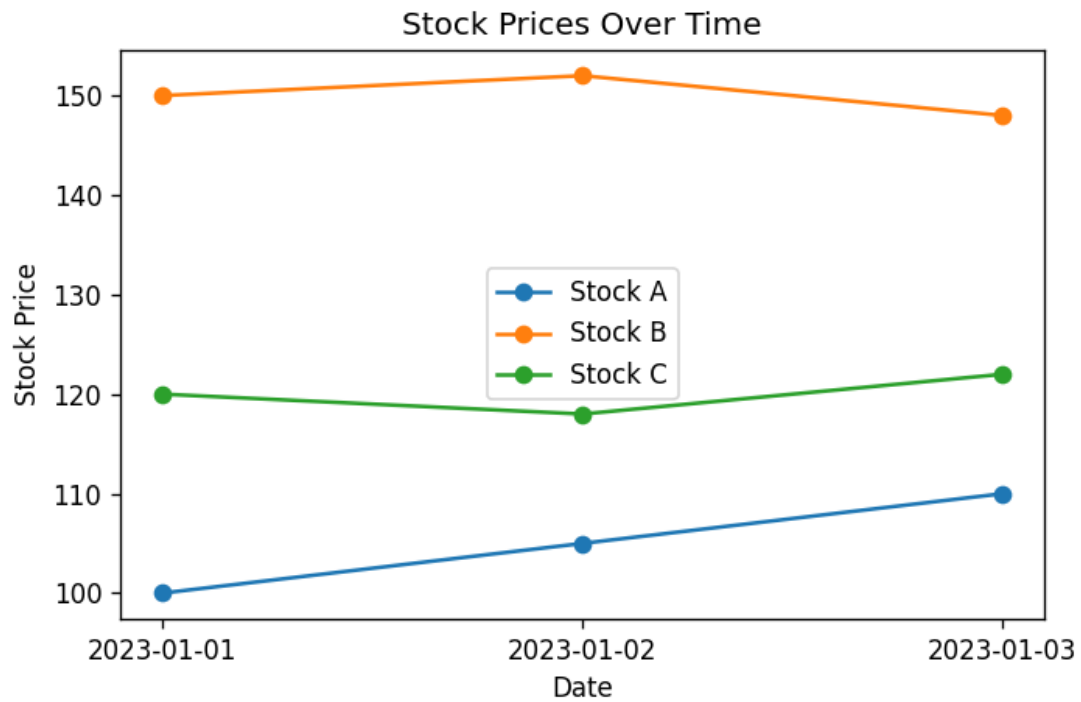
Q1. Line chart to visualize stock prices of three companies

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Date": ["2023-01-01", "2023-01-02", "2023-01-03"],
    "Stock A": [100, 105, 110],
    "Stock B": [150, 152, 148],
    "Stock C": [120, 118, 122]
})

plt.plot(df["Date"], df["Stock A"], marker="o", label="Stock A")
plt.plot(df["Date"], df["Stock B"], marker="o", label="Stock B")
plt.plot(df["Date"], df["Stock C"], marker="o", label="Stock C")
plt.xlabel("Date"); plt.ylabel("Stock Price")
plt.title("Stock Prices Over Time")
plt.legend()
plt.show()
```

Output:

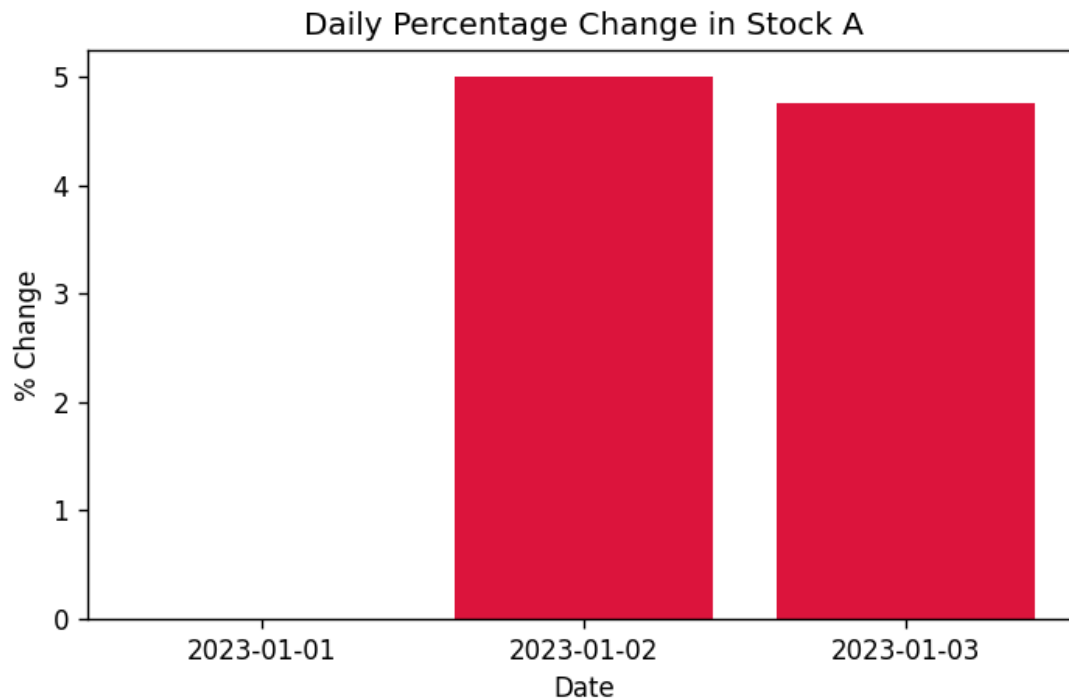


Set 20 Q1

Q2. Bar chart showing daily percentage change in Stock A

```
df["Stock A % Change"] = df["Stock A"].pct_change() * 100
plt.bar(df["Date"], df["Stock A % Change"].fillna(0), color="crimson")
plt.xlabel("Date"); plt.ylabel("% Change")
plt.title("Daily Percentage Change in Stock A")
plt.show()
```

Output:



Set 20 Q2

Q3. Table to display stock price data

```
print(df[["Date", "Stock A", "Stock B", "Stock C"]])
```

Output:

Stock Price Data

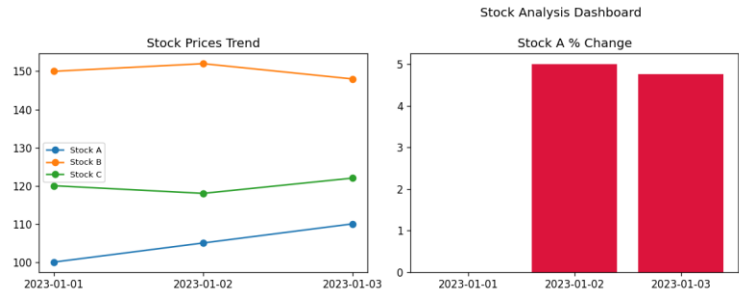
Date	Stock A	Stock B	Stock C
2023-01-01	100	150	120
2023-01-02	105	152	118
2023-01-03	110	148	122

Set 20 Q3

Q4. Dashboard combining line chart, bar chart and table

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))  
# axes[0]: line chart, axes[1]: bar chart, axes[2]: data table  
plt.suptitle("Stock Analysis Dashboard")  
plt.show()
```

Output:



Data Table

Date	Stock A	Stock B	Stock C
2023-01-01	100	150	120
2023-01-02	105	152	118
2023-01-03	110	148	122

Set 20 Q4